Lecture 3 — Complexity Analysis & Complete Search

> Year Curriculum · Prerequisites: Weeks 1–2 (I/O, STL) · Estimated time: 2 hours lecture + 4 hours practice

1. Overview

Before writing a line of code, a competitive programmer asks: *will this idea run in time?* **Complexity analysis** answers that question in seconds from the problem's constraints. The simplest algorithm design technique, **complete search** (brute force), works by trying every candidate. It is correct by construction, and the only question is whether it is fast enough.

This lecture covers:

1. Big-O notation and how to analyze loops, including logarithmic, harmonic, and amortized patterns.
2. Turning constraints into a time budget and a list of candidate techniques.
3. The three basic shapes of complete search: nested loops, subsets (bitmasks), and permutations.
4. Shrinking the search space by deriving variables, choosing what to enumerate, and pruning.
5. Using brute force as a correctness oracle for faster solutions.

Later weeks build on this one. Week 8 (backtracking) generalizes enumeration with recursion, Week 9 covers bitmasks in depth, and Week 26 (bitmask DP) speeds up exponential searches.

2. Big-O notation

We say $f(n) = O(g(n))$ if there are constants c and n_0 such that $f(n) \le c \cdot g(n)$ for all $n \ge n_0$. In practice:

- **drop constant factors**: $5n = O(n)$;
- **drop lower-order terms**: $n^2 + 100n + 7 = O(n^2)$;
- **logarithm bases do not matter**: $\log_2 n$ and $\log_{10} n$ differ by a constant.

Big-O describes how the running time **grows**. To estimate actual time, substitute the maximum input size and compare with the machine's throughput. In optimized C++, simple operations (arithmetic, array access, comparisons) run at roughly **10^8 to 10^9 per second**,

depending on the operation and memory access pattern. The safe planning number is **10^8 per second**.

Related notation: Θ means a tight bound (both upper and lower), and Ω means a lower bound. CP folklore says "O" even when it means Θ.

3. Analyzing code

3.1 Sequential and nested loops

Sequential blocks **add**: O(n) followed by O(n log n) is O(n log n). Nested loops **multiply** their iteration counts:

```
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
        work();     // runs n(n-1)/2 times -> O(n^2)
```

A triangular loop does half the work of a full square loop, but it is still $O(n^2)$.

3.2 Logarithmic loops

A loop that multiplies or divides its variable by a constant each iteration runs O(log n) times:

```
for (long long i = 1; i < n; i *= 2) ...   // ~log2(n) iterations
while (n > 0) { n /= 10; }                  // number of decimal digits
```

Binary search (Week 5), divide-and-conquer, and fast exponentiation (Week 11) get their logarithms this way.

3.3 The harmonic pattern

```
for (int i = 1; i <= n; i++)
    for (int j = i; j <= n; j += i)
        work();
```

The inner loop runs n/i times, so the total is $n \cdot (1 + 1/2 + 1/3 + \ldots + 1/n) = n \cdot H(n) \approx n \ln n =$ **O(n log n)**. This pattern computes all divisors of 1..n and underlies the sieve of Eratosthenes (Week 10, where the bound improves to O(n log log n)).

3.4 Amortized analysis

Sometimes one iteration is expensive but the **total** over all iterations is small:

```
int j = 0;
for (int i = 0; i < n; i++) {
    while (j < n && condition(i, j)) j++;
}
```

The inner `while` can run up to n times for a single `i`. But `j` never decreases and stops at n, so across the whole outer loop the inner body runs **at most n times in total**. The loop is O(n), not O(n²). This argument is the basis of two pointers (Week 6) and the monotonic stack (Week 12). It also explains why `vector::push_back` is O(1) amortized.

3.5 Recursion

Count the nodes of the recursion tree and multiply by the work per node. A function that calls itself twice on n − 1 has 2^n nodes. Merge sort calls itself twice on n/2 and does O(n) merging work, giving T(n) = 2T(n/2) + O(n) = O(n log n): there are log n levels, each costing O(n) in total.

3.6 Hidden costs

The complexity of a line includes the library calls on it:

- `s.substr(i, k)`, `string a = b`, and passing a `vector` by value: O(length).
- `v.erase(v.begin())`, `v.insert(v.begin(), x)`: O(n).
- `find(all(v), x)`: O(n). `set::find`: O(log n). `unordered_set::find`: O(1) average.
- `map` and `set` operations are O(log n) with a large constant from pointer-chasing tree nodes. Expect about 10^6–10^7 operations per second, not 10^8.

3.7 Memory

Memory limits are usually 256 MB. An `int` takes 4 bytes and a `long long` 8, so 256 MB holds about 6.4·10^7 ints. A 5000 × 5000 `int` table is 100 MB, which is fine. A 10^4 × 10^4 `int` table is 400 MB, which is not.

4. From constraints to algorithm

Assume ~10^8 operations per second and a 1–2 s limit:

Max n	Budget	Typical techniques
≤ 10–11	O(n!·n)	all permutations
≤ 20	O(2^n·n)	all subsets, bitmask DP
≤ 40	O(2^(n/2)·n)	meet in the middle

Max n	Budget	Typical techniques
≤ 100	$O(n^4)$	four nested loops
≤ 500	$O(n^3)$	triple loops, Floyd–Warshall, interval DP
≤ 5000	$O(n^2)$	double loops, quadratic DP
≤ 2·10^5 – 10^6	O(n log n)	sorting, binary search, heaps, segment trees
≤ 10^7 – 10^8	O(n)	linear scans, sieve
≤ 10^18	O(log n), $O(\sqrt{n})$	math, binary search on answer, fast power

Read this table backwards when designing. If n ≤ 2·10^5, you need roughly O(n log n), so ask what can be sorted, binary searched, or kept in a set. Small constraints are a **hint**, not an accident. A problem with n ≤ 18 almost always wants exponential search.

5. Complete search

Complete search enumerates the entire space of candidate answers, checks each, and keeps the valid or best one. Its strengths are correctness that is easy to argue, short code, and a full score when constraints are small. Its weakness is speed. The design question is always "**what do I enumerate, and how many candidates are there?**"

5.1 Shape 1: nested loops, and reducing dimensions

ABC085 C — Otoshidama. We need non-negative x, y, z with x + y + z = N and 10000x + 5000y + 1000z = Y, where N ≤ 2000. Three loops cost $2001^3 \approx 8 \cdot 10^9$ iterations, which is far too slow. But once x and y are chosen, z = N − x − y is **forced**. Two loops cost about 2·10^6 iterations, which is instant. One could reduce to a single loop with algebra, but two loops is already enough.

General rule: if a variable is determined by the others through an equation, compute it instead of enumerating it.

5.2 Shape 2: subsets with bitmasks

A subset of n items can be encoded as an n-bit integer `mask`, where bit i is 1 exactly when item i is in the subset. The integers 0 … 2^n − 1 then enumerate all 2^n subsets.

```
for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++)
        if (mask >> i & 1) { /* item i chosen */ }
}
```

Cost: O(2^n · n). With n = 20 that is about 2·10^7, which is fine. With n = 25 it is about 8·10^8, which is borderline. Beyond that, you need a different idea.

CSES 1623 — Apple Division. There are n ≤ 20 apples with weights up to 10^9. Divide them into two groups so that the difference of the group weights is minimal. Enumerate every subset as group 1. If its sum is s, the difference is |total − 2s|. Track the minimum. Use `long long`, since 20 · 10^9 = 2·10^10 > 2^31.

ABC128 C — Switches. There are N ≤ 10 switches and M ≤ 10 bulbs. Each bulb is connected to some switches and lights when the number of connected switches that are on has a given parity. Enumerate all 2^N on/off states and check each bulb in O(N·M) total per state.

CF 1097B — Petr and a Combination Lock. There are n ≤ 15 rotations by a_i degrees, each clockwise or counterclockwise. Can the lock end at 0? Enumerate the 2^n sign patterns and check whether the signed sum ≡ 0 (mod 360).

The art lies in **choosing what to enumerate**. In each example, a small set of binary decisions determines everything else.

5.3 Shape 3: permutations

When a problem asks for the best **order** of n ≤ 10 objects, enumerate all n! orders with `next_permutation`:

```
vector<int> p(n);
iota(p.begin(), p.end(), 0);          // start from the smallest permutation
do {
    // evaluate order p
} while (next_permutation(p.begin(), p.end()));
```

`next_permutation` rearranges the range into the next permutation in lexicographic order and returns `false` after the last one, wrapping back to sorted order. You must **start sorted** to visit all permutations. With repeated elements, it visits each **distinct** arrangement exactly once.

Cost: n! · (cost of evaluating one order). 10! = 3,628,800, and 10! · 10 ≈ 3.6·10^7 is fine. 12! ≈ 4.8·10^8 is too slow if each evaluation costs O(n).

CSES 1622 — Creating Strings. Print all distinct permutations of a string of length ≤ 8 in sorted order. Sort the string, then loop with `next_permutation`. The output count is at most 8! = 40320.

Shortest route through all cities (small n). Fix the start city and permute the other n − 1. For n = 9 that is 8! = 40320 routes. Week 26 solves n ≤ 20 using bitmask DP in O(2^n·n²), which shows how a smarter algorithm replaces brute force once n grows.

5.4 Shape 4 (preview): recursive generation and pruning

Loops and bitmasks generate candidates "blindly". Recursive generation builds a candidate one decision at a time, which lets you **prune**: abandon a partial candidate as soon as it cannot become valid or better than the best found so far. Examples: stop adding numbers once the sum exceeds the target, and stop placing queens once two attack each other. CSES 1624 (Chessboard and Queens) is the classic example and is the main topic of Week 8.

5.5 Making brute force faster

- **Derive, don't enumerate**, as in Otoshidama.
- **Enumerate the smaller side.** For "count pairs (a, b) with a·b ≤ N", enumerate a ≤ √N and count the matching b's by formula.
- **Exploit symmetry.** For unordered pairs, require i < j. For cyclic routes, fix the starting city.
- **Precompute** whatever the check needs (for example, the sum of each subset incrementally), so each candidate is cheap.
- **Meet in the middle** (for n ≤ 40): enumerate subsets of each half (2^20 each), sort one list, and binary search into it from the other. This is beyond this week, but keep it in mind.

6. Brute force as a testing oracle

Even when brute force is too slow for the full constraints, write it whenever your fast solution is tricky. Then **stress test**: generate thousands of small random inputs, run both programs, and stop at the first difference. That small failing input is far easier to debug than a hidden "WA on test 37". Week 35 builds a full stress-testing setup. For now, a shell loop is enough:

```
for i in $(seq 1 1000); do
  ./gen $i > in; ./brute < in > a; ./fast < in > b
  if ! cmp -s a b; then echo "diff on seed $i"; cat in; break; fi
done
```

7. Reference implementations

7.1 Otoshidama (ABC085 C)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    long long y;
    cin >> n >> y;
    for (int a = 0; a <= n; a++) {              // 10000-yen bills
```

```
        for (int b = 0; a + b <= n; b++) {      // 5000-yen bills
            int c = n - a - b;                   // 1000-yen bills are forced
            if (10000LL * a + 5000LL * b + 1000LL * c == y) {
                cout << a << ' ' << b << ' ' << c << '\n';
                return 0;
            }
        }
    }
    cout << "-1 -1 -1\n";
    return 0;
}
```

Time: $O(N^2) \approx 2 \cdot 10^6$ checks.

7.2 Apple Division (CSES 1623)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> p(n);
    long long total = 0;
    for (auto &x : p) { cin >> x; total += x; }
    long long best = LLONG_MAX;
    for (int mask = 0; mask < (1 << n); mask++) {
        long long s = 0;
        for (int i = 0; i < n; i++)
            if (mask >> i & 1) s += p[i];
        best = min(best, llabs(total - 2 * s));
    }
    cout << best << '\n';
    return 0;
}
```

Time: $O(2^n \cdot n) \approx 2 \cdot 10^7$ for $n = 20$. Mask and its complement give the same answer, so iterating only masks with bit $n-1$ cleared halves the work. That is an optional constant-factor improvement.

7.3 Creating Strings (CSES 1622)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;
    sort(s.begin(), s.end());
    vector<string> out;
```

```
    do {
        out.push_back(s);
    } while (next_permutation(s.begin(), s.end()));
    cout << out.size() << '\n';
    for (const auto &t : out) cout << t << '\n';
    return 0;
}
```

7.4 Shortest Hamiltonian path from city 0 (n ≤ 10), brute force

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<vector<long long>> d(n, vector<long long>(n));
    for (auto &row : d) for (auto &x : row) cin >> x;
    vector<int> order;                  // cities 1..n-1; city 0 is fixed first
    for (int i = 1; i < n; i++) order.push_back(i);
    long long best = LLONG_MAX;
    do {
        long long len = 0;
        int cur = 0;
        for (int v : order) { len += d[cur][v]; cur = v; }
        best = min(best, len);
    } while (next_permutation(order.begin(), order.end()));
    cout << best << '\n';
    return 0;
}
```

Time: O((n−1)! · n).

8. Common mistakes

1. **Skipping the estimate.** Coding an O(n²) idea for n = 2·10^5 and discovering the TLE after 30 minutes.
2. **`1 << n` for n ≥ 31.** That overflows `int`. Use `1LL << n` and a `long long` mask.
3. **Not starting `next_permutation` from sorted order.** You only get the permutations that come after the starting one.
4. **Overflow inside the check**, such as subset sums of large values. Use `long long`.
5. **Hidden linear costs** inside loops: copying strings, erasing from the front of a vector, `substr`.
6. **Too-large enumeration** when a variable could be derived.
7. **Trusting "about 10^9 is fine".** 10^9 simple operations takes 1–3 s or more. It might pass with a 2 s limit and very simple operations, but do not plan on it.

8. **Forgetting the empty subset or the full set** when the problem forbids them, for example "both groups must be non-empty".
9. **Deep recursion.** A recursion depth around 10^6 overflows the default stack.

9. Practice problems

CSES

- 1623 — Apple Division — https://cses.fi/problemset/task/1623
- 1622 — Creating Strings — https://cses.fi/problemset/task/1622
- 1624 — Chessboard and Queens (preview of Week 8) — https://cses.fi/problemset/task/1624

AtCoder

- ABC085 C — Otoshidama — https://atcoder.jp/contests/abc085/tasks/abc085_c
- ABC128 C — Switches — https://atcoder.jp/contests/abc128/tasks/abc128_c
- ABC051 B — Sum of Three Integers — https://atcoder.jp/contests/abc051/tasks/abc051_b
- ABC045 C — Many Formulas — https://atcoder.jp/contests/abc045/tasks/arc061_a
- ABC079 C — Train Ticket — https://atcoder.jp/contests/abc079/tasks/abc079_c

Codeforces

- 1097B — Petr and a Combination Lock — https://codeforces.com/problemset/problem/1097/B
- 550B — Preparing Olympiad — https://codeforces.com/problemset/problem/550/B
- 124B — Permutations — https://codeforces.com/problemset/problem/124/B

USACO

- 2019 December Bronze — Cow Gymnastics — http://www.usaco.org/index.php?page=viewproblem2&cpid=963
- 2018 January Bronze — Lifeguards — http://www.usaco.org/index.php?page=viewproblem2&cpid=784

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 2 ("Time complexity") and Chapter 5 ("Complete search"): https://cses.fi/book/book.pdf
- Steven Halim and Felix Halim, *Competitive Programming 4*, Section 3.2 ("Complete Search")
- USACO Guide, Bronze modules "Time Complexity", "Basic Complete Search", and "Complete Search with Recursion": https://usaco.guide/
- Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, Chapter 3 (asymptotic notation) and Chapter 16/17 (amortized analysis, depending on edition)